Phase 1: Full Stack CRUD Application

POC-07 — Inventory Management & Procurement System

Phase Weight: 15% | **Duration:** 5 working days | **Test Cases:** 20

---

1. Phase Overview

Build a production-quality inventory management platform for a retail business. Track products, monitor stock levels, raise purchase orders with suppliers, and get automated alerts when stock runs low.

---

2. Technology Setup

```
mkdir poc-07-inventory && cd poc-07-inventory
python -m venv venv && source venv/bin/activate
pip install fastapi uvicorn sqlalchemy pydantic pyjwt passlib structlog python-
dotenv pytest httpx
```

Environment (.env)

```
SECRET_KEY=your-secret-key
DATABASE_URL=sqlite:///./inventory.db
POC_ID=POC-07
```

---

3. User Stories

US-07-P1-01 — Product Registration

As Priya Sharma, **I want to** register a new product with SKU and reorder settings, **So that** it is tracked in the inventory system.

- **Given** valid product data | **When** POST /api/v1/products | **Then** SKU auto-generated as SKU-{CAT_PREFIX}-{NNNN}, HTTP 201

US-07-P1-02 — Stock Update with Movement Recording

As Dev Kumar, **I want to** update stock quantities by recording movements, **So that** all inventory changes are auditable.

- **Given** product_id and movement data | **When** PATCH /api/v1/products/{id}/stock | **Then** StockMovement created, quantity_on_hand updated, low_stock alert if below reorder_point

US-07-P1-03 — Purchase Order Creation

As Anita Singh, **I want to** raise purchase orders for low-stock products, **So that** stock is replenished from suppliers.

- **Given** supplier_id and product line items | **When** POST /api/v1/orders | **Then** PO created with auto-generated PO number, status="draft"

US-07-P1-04 — PO Receiving

As Dev Kumar, **I want to** mark a PO as received, **So that** stock levels update automatically.

- **Given** PO in submitted/acknowledged status | **When** PATCH /api/v1/orders/{id}/receive | **Then** status="received", StockMovement(receipt) created per item, stock updated

US-07-P1-05 — Low Stock Alerts

As Priya Sharma, **I want to** see all low-stock and out-of-stock products, **So that** I can trigger reorders before stockouts.

- **Given** authenticated manager | **When** GET /api/v1/stock/low-alerts | **Then** all products with quantity_available ≤ reorder_point returned, sorted by criticality

US-07-P1-06 — Supplier Catalog

As Anita Singh, **I want to** view a supplier's product catalog with cost prices, **So that** I can raise accurate POs.

- **Given** supplier_id | **When** GET /api/v1/suppliers/{id}/catalog | **Then** all products supplied by this supplier returned with unit_cost

US-07-P1-07 — Stock Movement History

As Raj Patel, **I want to** review stock movement history for any product, **So that** I can audit inventory changes.

- **Given** product_id | **When** GET /api/v1/products/{id} (include movements) | **Then** product details + recent movements returned

US-07-P1-08 — Inventory Dashboard

As Priya Sharma, **I want to** see a dashboard of overall inventory health, **So that** I can prioritize procurement actions.

- **Given** authenticated manager | **When** GET /api/v1/dashboard | **Then** total_products, low_stock_count, out_of_stock_count, open_po_count, total_stock_value returned

---

4. Database Models (app/models.py)

```
from sqlalchemy import Column, Integer, String, Float, Boolean, Date, DateTime,
Text, ForeignKey, Enum
from sqlalchemy.orm import relationship
from sqlalchemy.sql import func
from app.database import Base
```

```python
import enum

class Category(str, enum.Enum):
    grocery = "grocery"
    electronics = "electronics"
    clothing = "clothing"
    household = "household"
    personal_care = "personal_care"

CATEGORY_PREFIXES = {
    "grocery": "GRO", "electronics": "ELC", "clothing": "CLO",
    "household": "HHD", "personal_care": "PRC"
}

class MovementType(str, enum.Enum):
    receipt = "receipt"
    sale = "sale"
    adjustment = "adjustment"
    transfer = "transfer"
    returnm = "return"

class POStatus(str, enum.Enum):
    draft = "draft"
    submitted = "submitted"
    acknowledged = "acknowledged"
    received = "received"
    cancelled = "cancelled"

class Supplier(Base):
    __tablename__ = "suppliers"
    id = Column(Integer, primary_key=True)
    name = Column(String(200), nullable=False)
    supplier_code = Column(String(20), unique=True, nullable=False)
    contact_email = Column(String(200))
    payment_terms_days = Column(Integer, default=30)
    lead_time_days = Column(Integer, default=7)
    is_active = Column(Boolean, default=True)
    products = relationship("Product", back_populates="supplier")
    purchase_orders = relationship("PurchaseOrder", back_populates="supplier")

class Product(Base):
    __tablename__ = "products"
    id = Column(Integer, primary_key=True)
    sku = Column(String(20), unique=True, nullable=False)
    name = Column(String(200), nullable=False)
    category = Column(Enum(Category), nullable=False)
    unit_price = Column(Float, nullable=False)
    cost_price = Column(Float, nullable=False)
    unit_of_measure = Column(String(20), default="pieces")
    reorder_point = Column(Integer, default=10)
    reorder_quantity = Column(Integer, default=50)
    supplier_id = Column(Integer, ForeignKey("suppliers.id"), nullable=True)
    created_at = Column(DateTime, server_default=func.now())
    supplier = relationship("Supplier", back_populates="products")
```

```
    stock_level = relationship("StockLevel", back_populates="product",
uselist=False)
    movements = relationship("StockMovement", back_populates="product")
    alerts = relationship("StockAlert", back_populates="product")

class StockLevel(Base):
    __tablename__ = "stock_levels"
    id = Column(Integer, primary_key=True)
    product_id = Column(Integer, ForeignKey("products.id"), unique=True)
    quantity_on_hand = Column(Integer, default=0)
    quantity_reserved = Column(Integer, default=0)
    last_updated = Column(DateTime, server_default=func.now(),
onupdate=func.now())
    product = relationship("Product", back_populates="stock_level")

    @property
    def quantity_available(self):
        return max(0, self.quantity_on_hand - self.quantity_reserved)

class StockMovement(Base):
    __tablename__ = "stock_movements"
    id = Column(Integer, primary_key=True)
    product_id = Column(Integer, ForeignKey("products.id"))
    movement_type = Column(Enum(MovementType), nullable=False)
    quantity = Column(Integer, nullable=False)
    reference_number = Column(String(50), nullable=True)
    notes = Column(String(500), nullable=True)
    recorded_at = Column(DateTime, server_default=func.now())
    recorded_by = Column(String(100), default="system")
    product = relationship("Product", back_populates="movements")

class PurchaseOrder(Base):
    __tablename__ = "purchase_orders"
    id = Column(Integer, primary_key=True)
    po_number = Column(String(20), unique=True, nullable=False)
    supplier_id = Column(Integer, ForeignKey("suppliers.id"))
    status = Column(Enum(POStatus), default=POStatus.draft)
    total_amount = Column(Float, default=0.0)
    order_date = Column(Date, nullable=False)
    expected_delivery = Column(Date, nullable=True)
    received_date = Column(Date, nullable=True)
    created_at = Column(DateTime, server_default=func.now())
    supplier = relationship("Supplier", back_populates="purchase_orders")
    items = relationship("POItem", back_populates="purchase_order")

class POItem(Base):
    __tablename__ = "po_items"
    id = Column(Integer, primary_key=True)
    po_id = Column(Integer, ForeignKey("purchase_orders.id"))
    product_id = Column(Integer, ForeignKey("products.id"))
    quantity_ordered = Column(Integer, nullable=False)
    unit_cost = Column(Float, nullable=False)
    quantity_received = Column(Integer, nullable=True)
    purchase_order = relationship("PurchaseOrder", back_populates="items")
```

```
class StockAlert(Base):
    __tablename__ = "stock_alerts"
    id = Column(Integer, primary_key=True)
    product_id = Column(Integer, ForeignKey("products.id"))
    alert_type = Column(String(50), nullable=False)
    message = Column(String(500))
    is_resolved = Column(Boolean, default=False)
    triggered_at = Column(DateTime, server_default=func.now())
    product = relationship("Product", back_populates="alerts")

class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True)
    email = Column(String(200), unique=True)
    hashed_password = Column(String(200))
    full_name = Column(String(100))
    role = Column(String(50), default="staff")
    is_active = Column(Boolean, default=True)
```

---

5. Key Business Logic

SKU Generation

```
def generate_sku(category: str, db: Session) -> str:
    prefix = CATEGORY_PREFIXES.get(category, "GEN")
    count = db.query(Product).filter(Product.sku.like(f"SKU-{prefix}-%")).count()
    return f"SKU-{prefix}-{count + 1:04d}"
```

PO Number Generation

```
from datetime import date
def generate_po_number(db: Session) -> str:
    year = date.today().year
    count = db.query(PurchaseOrder).filter(
        PurchaseOrder.po_number.like(f"PO-{year}-%")
    ).count()
    return f"PO-{year}-{count + 1:04d}"
```

Stock Alert Check

```
def check_stock_alerts(product: Product, stock: StockLevel, db: Session):
    available = stock.quantity_available
    if available == 0:
        alert = StockAlert(product_id=product.id, alert_type="out_of_stock",
```

```
                            message=f"SKU {product.sku} is OUT OF STOCK.")
            db.add(alert)
        elif available <= product.reorder_point:
            alert = StockAlert(product_id=product.id, alert_type="low_stock",
                            message=f"SKU {product.sku}: only {available} units
left (reorder point: {product.reorder_point}).")
            db.add(alert)
```

PO Receive + Stock Update

```
def receive_purchase_order(po_id: int, db: Session):
    po = db.query(PurchaseOrder).filter(PurchaseOrder.id == po_id).first()
    po.status = POStatus.received
    po.received_date = date.today()
    for item in po.items:
        qty = item.quantity_received or item.quantity_ordered
        item.quantity_received = qty
        stock = db.query(StockLevel).filter(StockLevel.product_id ==
item.product_id).first()
        stock.quantity_on_hand += qty
        movement = StockMovement(
            product_id=item.product_id, movement_type=MovementType.receipt,
            quantity=qty, reference_number=po.po_number,
            notes=f"Received from PO {po.po_number}"
        )
        db.add(movement)
        product = db.query(Product).filter(Product.id == item.product_id).first()
        # Resolve low_stock alerts
        db.query(StockAlert).filter(
            StockAlert.product_id == item.product_id,
            StockAlert.is_resolved == False
        ).update({"is_resolved": True})
    db.commit()
```

---

6. Logging & Observability Requirements

```
logger.info("stock_updated", poc_id="POC-07", phase="P1",
            product_sku=..., movement_type=..., quantity=...,
new_quantity_on_hand=...)
logger.info("po_created", poc_id="POC-07", phase="P1",
            po_number=..., supplier_id=..., total_amount=...)
logger.info("low_stock_alert", poc_id="POC-07", phase="P1",
            product_sku=..., quantity_available=..., reorder_point=...)
```

---

7. Submission Checklist

- [ ] SKU auto-generated as SKU-{CAT_PREFIX}-{NNNN}
- [ ] PO number auto-generated as PO-{YEAR}-{NNNN}
- [ ] Stock alert created when quantity_available ≤ reorder_point
- [ ] PO receive endpoint updates stock and creates StockMovement records
- [ ] GET /api/v1/stock/low-alerts returns all products below reorder point
- [ ] GET /api/v1/dashboard returns inventory health summary
- [ ] Structured logging with poc_id="POC-07"
- [ ] 20 test cases: ≥14 passing